Lecture 04: Assignment 3 Problem Set & Systems Case Studies

Welcome to the official problem specification catalog for **Lecture 04: Strings, Lists, Tuples, Dictionaries, and Sets**. This document details the exact requirements, mathematical constraints, and algorithmic expectations for all 10 Assignment 3 questions and 3 systems engineering case studies.

📋 Table of Contents

- Problem 01: String Palindrome Invariance
- Problem 02: Arithmetic Mean of an Integer Sequence
- Problem 03: Dual List Ingestion & Ascending Sort
- Problem 04: Tuple Parity Partitioning
- Problem 05: Menu-Driven Student Gradebook Hash Table
- Problem 06: Lexical Word-Length Mapping
- Problem 07: Unicode Whitespace Frequency Analyzer
- Problem 08: Disjoint Set Verification
- Problem 09: Sequence Duplicate Detector
- Problem 10: Unique Character Extraction & Cardinality
- Systems Case Study 1: String Immutability & Buffer Concatenation Complexity
- Systems Case Study 2: Dynamic Array Growth Geometry & Amortized Allocation
- Systems Case Study 3: CPython Hash Table Architecture & Perturbation Probing

---

Problem 01: String Palindrome Invariance

Problem Description

Write a Python program that takes a string input from the user and determines whether it is a **palindrome**. A string is defined as a palindrome if it reads identically forwards and backwards under sequence reflection.

Mathematical Formulation

Let a string S of length N be an ordered sequence of characters:

$$S = (s_0, s_1, s_2, \ldots, s_{N-1})$$

The string S is invariant under reflection if and only if:

$$\forall i \in \{0, 1, \ldots, \lfloor N/2 \rfloor\}, \quad s_i = s_{N-1-i}$$

Requirements & Constraints

- Case Sensitivity: The comparison should preserve case by default, but your solution should optionally support case-insensitive and punctuation-stripped evaluation.
- Time Complexity Target: $\mathcal{O}(N)$ where N is the number of characters.
- Auxiliary Space Target: $\mathcal{O}(1)$ via two pointers or $\mathcal{O}(N)$ via sequence slicing.

Example I/O

- **Input:** `"radar"` $\implies$ **Output:** `True (The string 'radar' is a palindrome.)`
- **Input:** `"Python"` $\implies$ **Output:** `False (The string 'Python' is NOT a palindrome.)`

Problem 02: Arithmetic Mean of an Integer Sequence

Problem Description

Write a Python program that accepts a list of integers from the user and computes their arithmetic average (mean). The output must be displayed as a floating-point number formatted to two decimal places.

Mathematical Formulation

Given an integer sequence $X = [x_1, x_2, \ldots, x_N]$ of length $N \geq 1$, the arithmetic mean μ is defined as:

$$\mu = \frac{1}{N} \sum_{i=1}^{N} x_i$$

Requirements & Constraints

- Handle empty sequences gracefully without raising a `ZeroDivisionError`.
- Accept space-separated or comma-separated integers from console input.
- Time Complexity Target: $\mathcal{O}(N)$.
- Space Complexity Target: $\mathcal{O}(N)$ for list storage.

Example I/O

- **Input:** `[10, 20, 30, 40, 50]` $\implies$ **Output:** `Average: 30.00`
- **Input:** `[5, 12, 17, 24]` $\implies$ **Output:** `Average: 14.50`

Problem 03: Dual List Ingestion & Ascending Sort

Problem Description

Write a Python program that takes two separate lists of integers from the user, concatenates them into a single unified sequence, and sorts the combined list in ascending numerical order.

Mathematical Formulation

Given two ordered sets or multisets $A = [a_1, \ldots, a_N]$ and $B = [b_1, \ldots, b_M]$, compute the concatenated sequence:

$$C = A \parallel B = [a_1, \ldots, a_N, b_1, \ldots, b_M]$$

Then find the permutation π such that:

$$C_{\pi(1)} \leq C_{\pi(2)} \leq \cdots \leq C_{\pi(N+M)}$$

Requirements & Constraints

- Implement using list concatenation `+` or `.extend()`.
- Sort using Python's built-in Timsort algorithm (`sorted()` or `.sort()`).
- Time Complexity Target: $\mathcal{O}((N+M)\log(N+M))$.
- Space Complexity Target: $\mathcal{O}(N+M)$.

Example I/O

- **Input List 1:** `[4, 1, 7]`
- **Input List 2:** `[9, 2, 5]`

- **Output:** `Merged & Sorted: [1, 2, 4, 5, 7, 9]`

Problem 04: Tuple Parity Partitioning

Problem Description

Write a program to separate a given tuple of integers into two separate tuples: one containing all **even** numbers, and the other containing all **odd** numbers. Preserve the relative order of elements from the original tuple.

Mathematical Formulation

Given an input tuple:

$$T = (t_1, t_2, \ldots, t_N)$$

Partition T into two disjoint subsequences T_{even} and T_{odd} defined by the congruence relation:

$$T_{\text{even}} = (t_i \in T \mid t_i \equiv 0 \pmod 2)$$

$$T_{\text{odd}} = (t_i \in T \mid t_i \equiv 1 \pmod 2)$$

Requirements & Constraints

- Tuples are immutable; construction must produce distinct tuple objects.
- Time Complexity Target: $\mathcal{O}(N)$.
- Space Complexity Target: $\mathcal{O}(N)$.

Example I/O

- **Input Tuple:** `(10, 15, 22, 33, 40, 55, 62)`
- **Output Even:** `(10, 22, 40, 62)`
- **Output Odd:** `(15, 33, 55)`

Problem 05: Menu-Driven Student Gradebook Hash Table

Problem Description

Create an interactive, menu-driven Python program that maintains a record of student names and their corresponding letter grades using a dictionary. The program must repeatedly present four options until the user explicitly chooses to exit:

1. **Option A:** Add a new student and their grade.
2. **Option B:** Update the grade of an existing student.
3. **Option C:** Search for a student's grade by name.
4. **Option D:** Display all students and grades in formatted tabular form.
5. **Option E:** Exit application.

Mathematical Formulation

Let the gradebook be represented as a finite mapping:

$$f : \mathcal{K} \to \mathcal{V}$$

Where $\mathcal{K} \subset \text{String}$ represents the set of unique student identifiers, and $\mathcal{V} \in \{A, B, C, D, F\}$ represents academic grades.

Requirements & Constraints

- Handle key existence checks before updates or retrievals.
- Input sanitization (case-insensitive commands and names).
- Time Complexity Target: $\mathcal{O}(1)$ average for insertions, lookups, and updates.

Problem 06: Lexical Word-Length Mapping

Problem Description

Write a Python program that takes a list of words and constructs a dictionary where each word is a key, and its corresponding character length is the value. Demonstrate both a standard loop approach and a dictionary comprehension.

Mathematical Formulation

Given an alphabet Σ and a sequence of words $W = [w_1, w_2, \ldots, w_K]$ where $w_i \in \Sigma^*$, define the dictionary mapping:

$$D = \{w \mapsto |w| \mid w \in W\}$$

Example I/O

- **Input:** `["apple", "banana", "kiwi", "cherry"]`
- **Output:** `{'apple': 5, 'banana': 6, 'kiwi': 4, 'cherry': 6}`

Problem 07: Unicode Whitespace Frequency Analyzer

Problem Description

Write a Python program that takes a string input and counts the total number of whitespace characters contained within it. The count must include spaces (`' '`), tabs (`'\t'`), and newline characters (`'\n'`).

Mathematical Formulation

Given string $S = (s_1, s_2, \ldots, s_N)$, compute:

$$W(S) = \sum_{i=1}^{N} \mathbb{I}_{\text{whitespace}}(s_i)$$

Where the indicator function is:

$$\mathbb{I}_{\text{whitespace}}(c) = \begin{cases} 1 & \text{if } c \in \{' ', '\t', '\n', '\r', '\v', '\f'\} \\ 0 & \text{otherwise} \end{cases}$$

Example I/O

- **Input:** `"Python Data Science 2026\tModule\n"`
- **Output:** `Whitespace Count: 4`

Problem 08: Disjoint Set Verification

Problem Description

Write a Python program that takes two lists of elements and determines whether they are completely **disjoint** (i.e., they share zero common elements). Utilize set operations to achieve optimal asymptotic time complexity.

Mathematical Formulation

Given sequences L_1 and L_2, construct sets $S_1 = \text{set}(L_1)$ and $S_2 = \text{set}(L_2)$. The sequences are disjoint if and only if their intersection is the empty set:

$$S_1 \cap S_2 = \emptyset \iff |S_1 \cap S_2| = 0$$

Requirements & Constraints

- Time Complexity: $\mathcal{O}(\min(|S_1|, |S_2|))$ using `.isdisjoint()`.

Example I/O

- **Input 1:** `[1, 2, 3]`, `[4, 5, 6]` $\implies$ **Output:** `True (Lists are disjoint)`
- **Input 2:** `[1, 2, 3]`, `[3, 4, 5]` $\implies$ **Output:** `False (Shared element: {3})`

Problem 09: Sequence Duplicate Detector

Problem Description

Write a Python program that accepts a list containing potential duplicates and identifies all elements that appear more than once. Return the identified duplicate elements as a distinct set.

Mathematical Formulation

Given sequence $X = [x_1, \ldots, x_N]$, let the multiplicity of element v be:

$$m(v) = \sum_{i=1}^{N} \mathbb{I}(x_i = v)$$

Identify the set of duplicate elements $\mathcal{D}$:

$$\mathcal{D} = \{v \in X \mid m(v) > 1\}$$

Requirements & Constraints

- Achieve detection in single-pass $\mathcal{O}(N)$ time using tracking sets (`seen` and `duplicates`).
- Do NOT use nested loop $\mathcal{O}(N^2)$ count lookups.

Example I/O

- **Input:** `[1, 2, 3, 2, 4, 5, 1, 6]`
- **Output:** `Duplicates Found: {1, 2}`

Problem 10: Unique Character Extraction & Cardinality

Problem Description

Write a Python program that reads a string from the user, extracts all unique characters present in the string, and outputs both the collection of unique characters and its total cardinality.

Mathematical Formulation

Let string S be mapped to its constituent character set:

$$\mathcal{U} = \{c \mid c \in S\}$$

Compute the cardinality:

$$K = |\mathcal{U}|$$

Example I/O

- **Input:** `"engineering"`
- **Output Set:** `{'e', 'n', 'g', 'i', 'r'}`
- **Count:** `5`

Systems Case Study 1: String Immutability & Buffer Concatenation Complexity

Problem Statement

In Python, strings (`PyUnicodeObject`) are strictly immutable. Demonstrate empirically and theoretically why repeated string concatenation via the `+` operator inside a loop exhibits $\mathcal{O}(N^2)$ quadratic complexity, whereas preallocating a list of strings and joining them with `str.join()` runs in linear time $\mathcal{O}(N)$.

Technical Specifications

1. Implement both concatenation strategies for sequence sizes $N \in [10^3, 5 \times 10^4]$.
2. Profile the CPU elapsed execution time.
3. Formulate the mathematical proof of allocation summation:

$$T_{\text{naive}}(N) = \sum_{i=1}^{N} c \cdot i = c \cdot \frac{N(N+1)}{2} = \Theta(N^2)$$

4. Plot execution curves in high resolution (300 DPI).

Systems Case Study 2: Dynamic Array Growth Geometry & Amortized Allocation

Problem Statement

CPython's `list` is implemented as an array of pointers (`PyObject**`). When capacity is exceeded, CPython invokes `list_resize()` to allocate extra slots based on a proportional growth formula. Demonstrate the exact growth sequence, compute the waste factor, and prove why `list.append()` achieves $\mathcal{O}(1)$ amortized time complexity.

Technical Specifications

1. Track the internal allocated capacity of a Python list from size 0 to 100 using `sys.getsizeof()` .
2. Map the CPython 3.12+ over-allocation formula:

$$\text{allocated} = \text{new_size} + (\text{new_size} \gg 3) + (3 \text{ if new_size } < 9 \text{ else } 6)$$

3. Plot the step function of allocated slots versus current size.

Systems Case Study 3: CPython Hash Table Architecture & Perturbation Probing

Problem Statement

CPython dictionaries and sets utilize compact hash tables with open addressing and a pseudo-random perturbation sequence to resolve collisions. Analyze the collision resolution mechanism and determine the impact of hash collisions and load factor on lookup performance.

Technical Specifications

1. Compute probing sequence for hash collisions:

$$j_{i+1} = (5j_i + 1 + \text{perturb}) \pmod{2^k}$$

2. Show why CPython enforces resizing when the load factor reaches:

$$\alpha = \frac{\text{used}}{\text{size}} > \frac{2}{3}$$

3. Visualize probing trajectories under high and low collision conditions.